AD Aflevering 1

kvb125, fps618

Feb 2024

1 Mergesort

For at besvare dette spørgsmål, skal vi først forstå, hvordan MergeSort fungerer. MergeSort er en del og-hersk algoritme, der opdeler et array i to halvdele, sorterer dem rekursivt og til sidst fletter de to sorteret halvdele sammen. I hvert rekursivt kald opdeles arrayet yderligere, indtil vi når ned til enkelt element arrays, som per definition er sorteret.

1.1

For hvert niveau i træet fordobles antallet af kald, så for et fuldt balanceret binært træ med $\log_2(n)$ niveauer, vil det totale antal kald $T(n)$ være summen af alle kald over alle niveauer:

$$T(n) = 1 + 2 + 4 + \cdots + 2^{\log_2(n)-1}$$

Dette er en geometrisk serie med forholdet 2 , og summen kan findes ved hjælp af formlen for en endelig geometrisk serie:

$$S_n = \frac{a\,(1 - r^n)}{1 - r}$$

hvor a er det første led i serien (i dette tilfælde 1), r er forholdet mellem hvert led (i dette tilfælde 2), og n er antallet af led i serien. For MergeSort er antallet af led $\log_2(n)$, så vi får:

$$T(n) = \frac{1\left(1 - 2^{\log_2(n)}\right)}{1 - 2} = 2^{\log_2(n)} - 1 = n - 1$$

Men det er værd at bemærke, at denne formel kun tæller antallet af rekursive kald, der fører til en opdeling af arrayet, og inkluderer ikke de base cases, hvor funktionen ikke længere opdeler arrayet, men returnerer direkte (når størrelsen på det segment, der skal sorteres, er 1). For hvert blad i træet (hvor der er n blade i et fuldt balanceret binært træ), vil der være et ekstra kald, hvilket bringer det totale antal kald til:

$$T(n) = n - 1 + n = 2n - 1$$

For et array med 21 elementer ($n = 21$) vil det totale antal kald til MergeSort derfor være:

$$T(21) = 2 \times 21 - 1 = 41$$

2 Rekursionsligninger

For at afgøre, hvilke af disse rekursionsligninger der har løsningen $T(n) = \Theta\left(n^2\right)$, kan vi bruge Master Theoremet eller en direkte analyse, hvor det er relevant. Vi vil gennemgå hver rekursionsligning en efter en.

2.1 $T(n) = 4T(\lfloor n/2 \rfloor) + n \log n$

Denne ligning falder ind under Master Theoremet med $a = 4, b = 2,$ og $f(n) = n \log n$. Vi sammenligner $f(n)$ med $n^{\log_b a} = n^{\log_2 4} = n^2$: Da $f(n) = n \log n$ vokser langsommere end n^2, følger det, at denne rekursionsligning ikke har løsningen $T(n) = \Theta\left(n^2\right)$.

2.2 $T(n) = 4T(\lfloor n/2 \rfloor) + n^2$

Her er $a = 4, b = 2, \text{og } f(n) = n^2$. Så $n^{\log_b a} = n^2$: Da $f(n) = n^2$ matcher $n^{\log_b a}$, følger det fra Master Theoremet, at $T(n) = \Theta\left(n^2 \log n\right)$, hvilket ikke præcist er $\Theta\left(n^2\right)$, men tæt på. Dette er dog ikke et direkte match, så det kan være en kilde til forvirring; nøglen er at $\Theta\left(n^2 \log n\right)$ ikke reduceres til $\Theta\left(n^2\right)$.

2.3 $T(n) = 2T(\lfloor n/4 \rfloor) + n^2$

Igen, anvender vi Master Theoremet med $a = 2, b = 4, \text{og } f(n) = n^2$. Da $f(n)$ dominerer over $n^{\log_b a} = n^{\log 42} = n^{0.5}$, har vi $T(n) = \Theta\left(n^2\right)$.

2.4 $T(n) = 9T(\lfloor n/3 \rfloor) + n^3$

For denne ligning er $f(n) = n^3$, hvilket vokser hurtigere end $n^{\log 39} = n^2$, så denne har ikke løsningen $T(n) = \Theta\left(n^2\right)$.

2.5 $T(n) = T(n-1) + n^2$

Denne ligning kan løses direkte ved at summere n^2 fra 1 op til n, hvilket giver $\Theta\left(n^3\right)$ og ikke $\Theta\left(n^2\right)$

2.6 $T(n) = T(n-1) + n$

Denne ligning summer n fra 1 op til n, hvilket giver $\Theta\left(n^2\right)$ som løsning.

2.7 Konklusion:

Baseret på ovenstående analyse er de rekursionsligninger, der har løsningen $T(n) = \Theta\left(n^2\right)$, følgende:

$$T(n) = 2T(\lfloor n/4 \rfloor) + n^2$$
$$T(n) = T(n-1) + n$$

Disse to udtrykker en kvadratisk tidskompleksitet, enten direkte gennem summation eller via Master Theoremet.

3 Del og hersk

For at formulere rekursionsligningen for køretiden $T(n)$ af algoritmen A baseret på beskrivelsen af, hvordan A fungerer, kan vi opdele analysen i de tre hoveddele af algoritmens udførelse:

1. Forbehandlingstid: Tiden det tager at beregne de fire inputs X_1, X_2, X_3, X_4, som er $O(n \log n)$.
2. Rekursiv Beregning: De fire rekursive kald til A, hver på input af størrelse $\lfloor n/2 \rfloor$.
3. Efterbehandlingstid: Tiden det tager at kombinere de rekursive svar til et output $A(X)$, som er $O\left(n^2\right)$.

Ud fra dette kan vi formulere følgende rekursionsligning for $T(n)$:

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n \log n) + O\left(n^2\right)$$

Her betegner $4T\left(\frac{n}{2}\right)$ de fire rekursive kald på halverede inputstørrelser, $O(n \log n)$ repræsenterer forbehandlingstiden, og $O\left(n^2\right)$ repræsenterer efterbehandlingstiden for at kombinere de rekursive resultater.

3.1 Anvendelse af Master Theoremet

For at anvende Master Theoremet skal vi først identificere termerne a, b, og $f(n)$ fra rekursionsligningen, hvor $T(n) = aT\left(\frac{n}{b}\right) + f(n)$, og sammenligne $f(n)$ med $n^{\log_b a}$.

I vores tilfælde:
- $a = 4$, da der er fire rekursive kald.
- $b = 2$, da hver rekursion opererer på halvdelen af inputstørrelsen.
- $f(n) = O(n \log n) + O\left(n^2\right)$.

Da det dominerende led i $f(n)$ er $O\left(n^2\right)$, vil vi fokusere på dette for vores analyse med Master Theoremet.

Vi skal sammenligne $f(n)$ med $n^{\log_b a} = n^{\log_2 4} = n^2$. Her er $f(n) = O\left(n^2\right)$, hvilket matcher formen $n^{\log_b a}$, hvilket indikerer, at vi er i det tredje tilfælde af Master Theoremet, hvor $f(n)$ er asymptotisk lige så stort som $n^{\log_b a}$.

3.2 Konklusion

Ifølge det tredje tilfæIde af Master Theoremet, hvis $f(n) = \Theta\left(n^{\log_b a}\right)$, så er $T(n) = \Theta(f(n)) = \Theta\left(n^2\right)$, under forudsætning af at visse regelmæssighedsbetingelser er opfyldt (hvilket typisk indebærer, at $f(n)$ ikke vokser "for hurtigt" i forhold til $n^{\log_b a}$).

Derfor, baseret på den givne beskrivelse af algoritmen A og anvendelsen af Master Theoremet, kan vi konkludere, at køretiden $T(n)$ for algoritmen A på input X er $T(n) = \Theta\left(n^2\right)$.

4 Køretid

For at analysere køretiden $T(n)$ for pseudokoden 'LOOPS (n)' , skal vi se på, hvordan de indlejrede løkker fungerer og opsummere antallet af gange, operationen $r = r + j$ udføres.

Den yderste løkke kører fra 1 til $n/2$, hvilket giver os $n/2$ iterationer. Den indre løkke starter ved $j = n/2 - i$ og kører til n, hvor i er den aktuelle iteration af den yderste løkke. Antallet af iterationer i den indre løkke afhænger derfor af værdien af i.

For at bestemme den samlede køretid, $T(n)$, skal vi bestemme antallet af iterationer af den indre løkke for hver værdi af i: - For $i = 1$, kører den indre løkke fra $j = n/2 - 1$ til n, hvilket er lidt mindre end $n/2$ iterationer.
- For $i = 2$, kører den indre løkke fra $j = n/2 - 2$ til n, og så videre.

Antallet af iterationer i den indre løkke øges for hver værdi af i, op til $n/2$. Vi vil nu beregne den samlede køretid ved at summere iterationerne for den indre løkke over alle iterationerne af den yderste løkke.

4.1 Køretidsanalyse

Antallet af iterationer for den indre løkke som funktion af i starter ved cirka $n/2$ og stiger til næsten n i de senere iterationer. For at simplificere analysen kan vi antage, at den indre løkke i gennemsnit kører $\frac{n/2+n}{2}$ gange for hver iteration af den yderste løkke, som er en grov tilnærmelse.

Antallet af gange $r = r + j$ udføres er derfor:

$$\frac{n}{2} \times \frac{3n/4}{=} \frac{3n^2}{8}$$

Dette viser, at den samlede køretid $T(n)$ har en kvadratisk afhængighed af n, hvilket betyder:

- $T(n) = O\left(n^2\right)$ fordi den vre grænse for køretiden vokser kvadratisk med n.
- $T(n) = \Omega(n)$ da køretiden mindst vokser lineært med n.
- $T(n) = \Omega(n \log n)$ og $T(n) = O(n \log n)$ Generelt, hvis $T(n) = O\left(n^2\right)$, kan det antages at $T(n) = O(n \log n)$ også gælder, da n^2 vokser hurtigere end $n \log n$. Dog, uden en direkte lineær eller $n \log n$ vækst bevist, er det mere nøjagtigt kun at bekræfte de direkte følger af den kvadratiske vækst.

5 Invarianter

For at bestemme gyldige løkke-invarianter for funktionen `Second-Smallest`, analyserer vi udsagnene ved starten af hver iteration i `for`-løkken:

5.1

1. $m = \min(A[1], \ldots, A[i-1])$: Dette udsagn er **gyldigt** som en løkke-invariant. Ved starten af hver iteration repræsenterer m det mindste element blandt de første $i - 1$ elementer i arrayet A, hvilket opretholdes gennem hver iteration.

2. $m = \min(A[1], \ldots, A[n])$: Dette udsagn er **ugyldigt** som en løkke-invariant, da m kun vil være minimummet af hele arrayet A efter alle iterationer er gennemført, ikke nødvendigvis ved starten af hver iteration.

3. $r \geq m$: Dette udsagn er **gyldigt** som en løkke-invariant. Variablen r vil altid være større end eller lig med m, da r enten er uendelig eller værdien af det næstmindste element fundet indtil nu.

4. $m \geq 0$: Dette udsagn er **ugyldigt** som en løkke-invariant, da arrayet A kan indeholde negative tal, hvilket betyder, at m potentielt kan være negativ.

5. Mindst 1 element i $A[1], \ldots, A[i-1]$ er mindre end eller lig med r: Dette udsagn er **gyldigt** som en løkke-invariant, da r vil være uendelig eller det næstmindste element blandt de første $i-1$ elementer, hvilket sikrer, at mindst ét element opfylder betingelsen.

6. Højst 2 elementer i $A[1], \ldots, A[i-1]$ er mindre end eller lig med r: Dette udsagn er **gyldigt** som en løkke-invariant, da m og r repræsenterer det mindste og næstmindste element fundet, hvilket begrænser antallet af elementer, der kan være mindre end eller lig med r til højst to.

Baseret på ovenstående analyse er de gyldige løkke-invarianter: 1, 3, 5 og 6.

6 Induktionsbeviser

For at bevise, ved induktion, at antallet af diagonaler d_n i en regulær n-kant er givet ved formlen $d_n = \frac{n(n-3)}{2}$ for $n \geq 3$, følger vi induktionsprincippet i to trin: basistrinnet og induktionstrinnet.

6.1 Basistrin

Først viser vi, at udsagnet gælder for $n = 3$, det mindste antal sider for en polygon (en trekant), hvor der forventes 0 diagonaler, da alle hjørner er naboer, og ingen linjer kan tegnes, der fuldt ud ligger inde i figuren uden at krydse en kant.

Indsæt $n = 3$ i formlen:

$$d_3 = \frac{3(3-3)}{2} = \frac{3 \cdot 0}{2} = 0$$

Udsagnet holder for basistrinnet, da en trekant ikke har nogen diagonaler.

6.2 Induktionstrin

Vi antager, at udsagnet gælder for en k-kant, så $d_k = \frac{k(k-3)}{2}$. Dette er induktionsantagelsen. Vi skal nu bevise, at udsagnet også gælder for en $(k+1)$-kant, hvilket vil sige, at $d_{k+1} = \frac{(k+1)((k+1)-3)}{2}$.

Når vi tilføjer et ekstra hjørne (og dermed en ekstra side) til en k-kant for at lave en $(k+1)$-kant, gør vi følgende:

1. Vi tilføjer $k-1$ nye diagonaler: den nye hjørne forbinder til hvert af de eksisterende hjørner undtagen sine to naboer og sig selv.
2. Vi tilføjer 1 ny kant, som ikke bidrager til antallet af diagonaler direkte.

Derfor øges antallet af diagonaler med $k-1$, når vi går fra k til $k+1$. Ud fra induktionsantagelsen har vi:

$$d_k = \frac{k(k-3)}{2}$$

For at finde d_{k+1}, tilføjer vi $k-1$ til d_k :

$$d_{k+1} = d_k + (k-1)$$
$$d_{k+1} = \frac{k(k-3)}{2} + (k-1)$$

Vi udvider og simplificerer udtrykket:

$$
\begin{aligned}
d_{k+1} &= \frac{k(k-3)+2(k-1)}{2} \\
d_{k+1} &= \frac{k^2-3k+2k-2}{2} \\
d_{k+1} &= \frac{k^2-k-2}{2} \\
d_{k+1} &= \frac{k^2-k+2k-2}{2} \text{ (tilføjer og trækker } 2k \text{)} \\
d_{k+1} &= \frac{(k+1)(k-2)}{2} \\
d_{k+1} &= \frac{(k+1)((k+1)-3)}{2}
\end{aligned}
$$

Dette viser, at hvis udsagnet gælder for en k-kant, gælder det også for en $(k+1)$-kant.

6.3 Konklusion

Da både basistrinnet og induktionstrinnet er opfyldt, følger det ved induktion, at formlen $d_n = \frac{n(n-3)}{2}$ gælder for antallet af diagonaler i en regulær n-kant for alle $n \geq 3$.

7 Korrekthed og køretid

a) Matrix-vektor produktet

For at vise, at for $x = (F_{n-2}, F_{n-1})^T$ opfylder matrix-vektor produktet ligheden $Ax = (F_{n-1}, F_n)^T$ for $n > 1$, kan vi beregne produktet direkte:

$$
Ax = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} F_{n-2} \\ F_{n-1} \end{pmatrix} = \begin{pmatrix} F_{n-1} \\ F_{n-2} + F_{n-1} \end{pmatrix} = \begin{pmatrix} F_{n-1} \\ F_n \end{pmatrix}
$$

Denne udregning viser, at når vi anvender matrix A på vektoren x, opnår vi en ny vektor, hvor det første element er det oprindelige andet element (F_{n-1}), og det andet element er summen af de oprindelige to elementer ($F_{n-2} + F_{n-1} = F_n$), hvilket er definitionen af Fibonacci-tallene.

b) Bevis for $A^{n-1}x = (F_{n-1}, F_n)^T$

For at bevise $A^{n-1}x = (F_{n-1}, F_n)^T$ for $x = (0,1)^T$ og $n \geq 1$, kan vi bruge induktion: - Basis trin: For $n = 1$, har vi $A^0 x = x = (0,1)^T = (F_0, F_1)^T$, hvilket passer med definitionen af Fibonacci-tallene. - Induktionsantagelse: Antag at $A^{k-1}x = (F_{k-1}, F_k)^T$ holder for et vilkårligt k. - Induktionstrin: Vi skal vise, at hvis $A^{k-1}x = (F_{k-1}, F_k)^T$, så gælder $A^k x = (F_k, F_{k+1})^T$. Ved at anvende A på $A^{k-1}x$, får vi:

$$
A^k x = A\left(A^{k-1}x\right) = A\left(F_{k-1}, F_k\right)^T = \left(F_k, F_{k+1}\right)^T
$$

Dette beviser ved induktion, at $A^{n-1}x = (F_{n-1}, F_n)^T$ for alle $n \geq 1$.

c) Beregning af A^{n-1}i$O(\log n)$ tid

For at udregne A^{n-1}i$O(\log n)$ tid, kan vi anvende eksponentiering ved kvadrering, som er en effektiv måde at beregne potenser på. Idéen er at udnytte, at:

$$
A^{2k} = \left(A^k\right)^2 \quad \text{og} \quad A^{2k+1} = A\left(A^k\right)^2
$$

Dette betyder, at vi kan opdele problemet i mindre dele, og hver gang vi halverer eksponenten, reducerer vi antallet af nødvendige multiplikationer betydeligt. Eksponentiering ved kvadrering kræver kun $O(\log n)$ matrixmultiplikationer.

d) Beregning af F_n i $O(\log n)$ tid

Ud fra del (c) kan vi konkludere, at siden $A^{n-1}x = (F_{n-1}, F_n)^T$ kan beregnes i $O(\log n)$ tid ved at anvende eksponentiering ved kvadrering, og da aritmetiske operationer antages at tage konstant tid, kan vi beregne F_n (som er det andet element i vektoren $A^{n-1}x\big)$ i $O(\log n)$ tid.